

Phishing detection with fewer clues

WHAT WE ARE MAKING

A program that looks at a website link and says whether it is a **scam or safe**. It never opens the page — it only measures things about the link, like how many dots it has, how long the domain is, and when the domain expires.

The data gives us **111 things to check**. That is far more than needed, and each one costs time. So we use a swarm algorithm to find the **~25 checks that actually matter**, and show the smaller version still works just as well.

THE DATA

FILE	dataset_small.csv
ROWS	58,645 websites
COLUMNS	112 (111 features + 1 answer)
ANSWER	phishing — 0 = safe, 1 = scam
COST	Free, no login

OUR 3 OBJECTIVES

1. Build the model using **all 111 features** — our starting point.
2. Use **PSO** to find the smallest set of features that still works.
3. Show the smaller model is **faster and just as accurate**.

HOW PSO WORKS

Like a flock of birds looking for food. Each bird remembers its own best spot, and knows the flock's best spot, and moves towards both. Slowly they all gather somewhere good.

Here each **bird** = **111 on/off switches**, one per feature. We score a bird by training the model with only its switched-on features.

$$\text{score} = 0.9 \times (1 - \text{accuracy}) + 0.1 \times (\text{features kept} \div 111)$$

Lower is better. The first half punishes being wrong, the second half punishes keeping too many. **Without that second half, PSO would keep all 111 forever.**

Why not just test every combination? There are 2^{111} of them — about 2.6 followed by 33 zeros. No computer can. Settling for a very good answer because a perfect one is impossible *is* soft computing.

WHY PSO, NOT PCA

PCA blends features into new numbers nobody can read.

PSO gives back **real feature names** — an actual list of checks a security team could use. That is a real advantage, not a consolation prize.

We still run PCA, a filter method, and a Genetic Algorithm just to compare against.

4 RULES WE MUST NOT BREAK

1. **Split train/test BEFORE running PSO.** If PSO sees the test data, every result is fake.
2. Run PSO **5–10 times** and report the average. One run is just luck.
3. Report **recall**, not only accuracy. Missing a scam is worse than a false alarm.
4. Handle the case where PSO picks **zero features**, or it crashes.

OPTIONAL — ONLY IF WE WANT MORE

✗ Merging two datasets — bad idea

The columns don't match, and the two sets measure things differently. Worse, the model can secretly learn *"which dataset is this from?"* instead of *"is this a scam?"* — that looks like great accuracy but is fake, and examiners check for it.

✓ Cross-dataset validation — good option

Run the same method on **both datasets separately**, then compare. No merging. It answers a real question — does our method work everywhere, or did it just fit one dataset? This is what real research papers do.

TARGET RESULT

95–97% accuracy while keeping only **20–30 of the 111 features**.

If we somehow get 99.9%, something is wrong — check rule 1 first.

Tools: Python · pandas · scikit-learn · pyswarms. Runs free in Google Colab. **No hardware needed.**

LINKS

Dataset — start here

github.com/GregaVrbancic/Phishing-Dataset

The file we use

raw.githubusercontent.com/.../dataset_small.csv

Second dataset (for comparison)

archive.ics.uci.edu/dataset/327/phishing+websites

PSO tutorial

pyswarms.readthedocs.io/.../feature_subset_selection